Remember, as OpenTelemetry matures and logs continue stabilizing, make sure you have a migration strategy in place to move off of whatever log routing agent you are currently using with the OpenTelemetry collector. Let's now move on to metrics.

4.2 Metrics collection

In contrast to logs, the landscape is less fragmented concerning metrics. Over the past years (effectively, with the rise of Kubernetes, since 2016), the dominant player is the Prometheus project, which put forward both the way metrics are collected as well as the way metrics are represented on the wire—the *exposition format*. Let's first have a look at Prometheus and then take a quick peek at other established agents in the metrics space.

> **NOTE** While Prometheus established pull-based collection—that is, the agent queries the metrics source via an HTTP endpoint defaulting to `/metrics`—the situation is more nuanced in the context of OpenTelemetry. There, the ingress part is still pull based; however, the egress part (toward the backend destination), is typically push based. We will get into the details in the next section on OpenTelemetry.

4.2.1 Prometheus

Prometheus (https://prometheus.io/) is a graduated CNCF project that grew up in tandem with Kubernetes, where it also is predominantly used. It covers the ingestion, storing, and query and federation interfaces for metrics, including a simple UI. Further, part of the Prometheus project is alerting capabilities (a separate binary but tightly integrated).

In chapter 2, we had a general discussion about metrics. Now, let's have a closer look at how Prometheus does things, in greater detail.

A time series in Prometheus is uniquely identified by its metric name and a number of (optional) key–value pairs referred to as *labels*. A metric name captures the general feature you want to measure. For example, `api_requests_total` could be used for the total number of API invocations. Labels, on the other hand, represent Prometheus's dimensional data model. Let's say you want to capture two dimensions with the API invocation metric: the HTTP method used and what the URL path was. You could use the two labels `method` and `path` for this, resulting in the following, for example:

```
api_requests_total{method="GET", path="/some/interesting/thing"}
```

There are different types of metrics Prometheus supports, including counters, gauges, and histograms, and while Prometheus has naming conventions for metrics and labels (https://prometheus.io/docs/practices/naming/), it doesn't define or enforce semantics for them. The lack of defined semantics in Prometheus is, in practice, not a huge problem, since Prometheus offers a feature called *relabeling* (https://prometheus.io/docs/prometheus/latest/configuration/configuration/#relabel_config), which allows you to standardize on naming.